

Module 1

Key Takeaways

- Understood and applied SOLID principles for building maintainable and scalable software.
- Explored design patterns such as Singleton, Factory Method, and Adapter to solve common design problems.
- Enhanced code reusability, flexibility, and overall software design through practical implementation.

Exercise 1: Implementing the Singleton Pattern Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Steps:

1. Create a New Java Project:

- Create a new Java project named **SingletonPatternExample**.

2. Define a Singleton Class:

- Create a class named **Logger** that has a private static instance of itself.
- Ensure the constructor of **Logger** is private.
- Provide a public static method to get the instance of the **Logger** class.

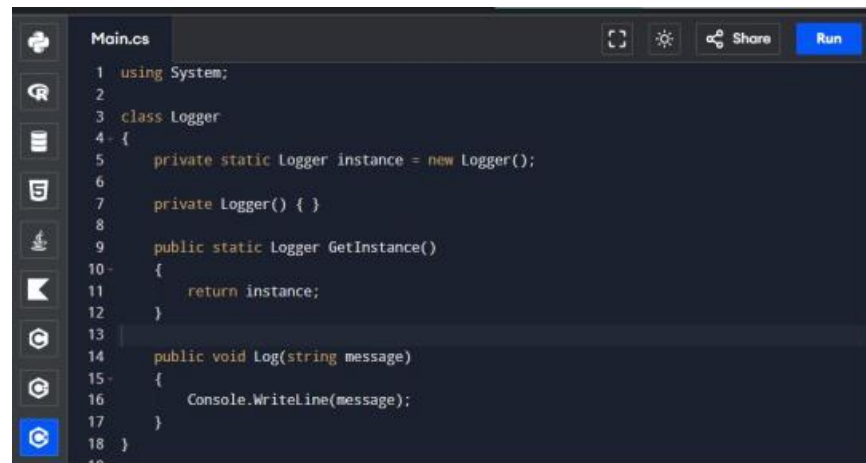
3. Implement the Singleton Pattern:

- Write code to ensure that the **Logger** class follows the Singleton design pattern.

4. Test the Singleton Implementation:

- Create a test class to verify that only one instance of **Logger** is created

Code :

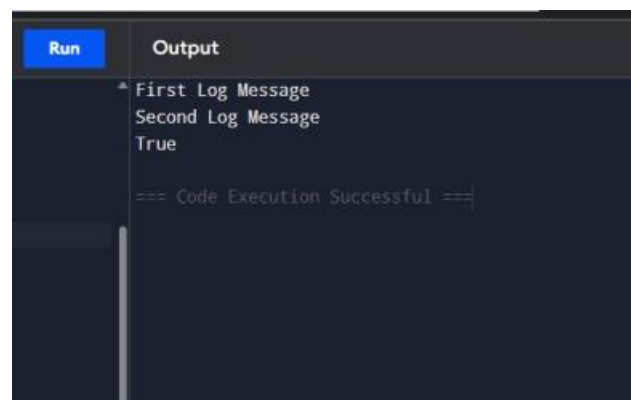


```
1 using System;
2
3 class Logger
4 {
5     private static Logger instance = new Logger();
6
7     private Logger() { }
8
9     public static Logger GetInstance()
10    {
11        return instance;
12    }
13
14    public void Log(string message)
15    {
16        Console.WriteLine(message);
17    }
18 }
```



```
19
20 class Program
21 {
22     static void Main(string[] args)
23     {
24         Logger logger1 = Logger.GetInstance();
25         Logger logger2 = Logger.GetInstance();
26
27         logger1.Log("First Log Message");
28         logger2.Log("Second Log Message");
29
30         Console.WriteLine(logger1 == logger2);
31     }
32 }
```

OUTPUT :



```
Run Output
First Log Message
Second Log Message
True

=== Code Execution Successful ===
```

Exercise 2: Implementing the Factory Method Pattern Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

Steps:

1. Create a New Java Project:

- Create a new Java project named **FactoryMethodPatternExample**.

2. Define Document Classes:

- Create interfaces or abstract classes for different document types such as **WordDocument**, **PdfDocument**, and **ExcelDocument**.

3. Create Concrete Document Classes:

- Implement concrete classes for each document type that implements or extends the above interfaces or abstract classes.

4. Implement the Factory Method:

- Create an abstract class **DocumentFactory** with a method **createDocument()**.
- Create concrete factory classes for each document type that extends **DocumentFactory** and implements the **createDocument()** method.

5. Test the Factory Method Implementation:

Create a test class to demonstrate the creation of different document types using the factory method.

Code :

```
Main.cs
1  using System;
2  interface IDocument
3  {
4      void Open();
5  }
6  class WordDocument : IDocument
7  {
8      public void Open()
9      {
10         Console.WriteLine("Word Document Opened");
11     }
12 }
13 class PdfDocument : IDocument
14 {
15     public void Open()
16     {
17         Console.WriteLine("PDF Document Opened");
18     }
19 }
20 class ExcelDocument : IDocument
21 {
22     public void Open()
23     {
24         Console.WriteLine("Excel Document Opened");
25     }
26 }
```

Output :

```
Output
Word Document Opened
PDF Document Opened
Excel Document Opened

=== Code Execution Successful ===
```

